

Phase 9: LLM Client

Hospital Patient Helpdesk Chatbot

Python module: 06_rag_pipeline/09_llm_client.py **Jupyter notebook:**

13_notebooks/09_llm_client.ipynb **Purpose:** Connect safe, citation-ready Phase 8 prompts to an offline test provider, OpenAI, Google Gemini, Anthropic Claude, or a local Ollama model.

1. Phase objective

Phase 9 creates a provider-neutral language-model boundary for the RAG pipeline. It accepts the prompt bundles produced in Phase 8 and returns a consistent response schema regardless of provider. This keeps the future RAG chain independent of vendor-specific response formats.

The default provider is offline. It is deterministic, private, free to run, and designed for pipeline validation. It does not claim to be a general-purpose LLM. Hosted providers are opt-in and require an explicit model name and credentials.

2. Folder placement

```
hospital_patient_helpdesk_chatbot/
|-- 01_data/
|   |-- processed/
|       |-- 08_prompt_bundles.json
|       |-- 09_llm_responses.json
|       |-- 09_llm_report.json
|       |-- 09_llm_audit.csv
|       |-- 09_failed_llm_requests.json
|       |-- plots/
|           |-- 09_llm_latency_by_prompt.png
|           |-- 09_llm_output_length_distribution.png
|-- 06_rag_pipeline/
|   |-- 09_llm_client.py
|   |-- 09_llm_client.md
|   |-- 09_llm_client.pdf
|-- 13_notebooks/
|   |-- 09_llm_client.ipynb
|-- .env
|-- README.md
```

All Phase 9 generated artifacts begin with 09_ so their source phase is immediately clear.

3. Input files

01_data/processed/08_prompt_bundles.json

The required input is a JSON list generated by Phase 8. Every item contains:

Field	Purpose
prompt_id	Stable identifier used in outputs and audit logs.
mode	Safety route such as grounded, emergency, or unsafe advice.
question	Original patient helpdesk question.
system_prompt	Hospital grounding and safety instructions.
user_prompt	Retrieved context and user request.
sources	Approved evidence and citation labels.
safety_labels	Signals propagated from earlier phases.

estimated_prompt_tokens	Approximate input size for monitoring.
-------------------------	--

.env

Configuration values are read from .env, then overridden by non-empty process environment variables. Example:

```
LLM_PROVIDER=offline
OPENAI_API_KEY=
OPENAI_MODEL=
GEMINI_API_KEY=
GEMINI_MODEL=
ANTHROPIC_API_KEY=
ANTHROPIC_MODEL=
OLLAMA_MODEL=
```

API keys are never written to JSON, CSV, plots, or console output.

4. Output files

Numbered artifact	Description
09_llm_responses.json	Provider-neutral response records for Phase 10.
09_llm_report.json	Provider, model, counts, mode totals, latency summary, and artifact inventory.
09_llm_audit.csv	One compact operational row per successful response.
09_failed_llm_requests.json	Prompt IDs and sanitized error messages for failed requests.
09_llm_latency_by_prompt.png	Per-prompt request latency.
09_llm_output_length_distribution.png	Distribution of response character counts.

No plot is required for correctness. The two plots are useful for spotting latency spikes, unusually short answers, excessive verbosity, and possible output truncation.

5. Normalized response schema

Each item in 09_llm_responses.json contains:

```
{
  "prompt_id": "PROMPT-001",
  "provider": "offline",
  "model": "offline-grounded-v1",
  "mode": "grounded_answer",
  "text": "Verified hospital answer [S1]",
  "citations": ["[S1]"],
  "finish_reason": "stop",
  "latency_ms": 0.1,
  "input_tokens": 500,
  "output_tokens": 20,
  "request_id": null,
  "safety_labels": []
}
```

Token fields may be null if a provider does not return usage information. The stable schema prevents downstream code from depending on a provider's native payload.

6. Python module code sections

Configuration and data contracts

ClientConfig, LLMResponse, and BatchResult define validated, typed boundaries. Invalid providers, empty model names, non-positive timeouts, too-small output limits, and negative retry counts fail early.

Environment loading

`load_env_file()` reads simple KEY=VALUE entries without requiring `python-dotenv.environment()` overlays process variables so deployment secrets can override local development settings.

Prompt input validation

`load_prompt_bundles()` confirms that the Phase 8 file exists, is a JSON list, and includes all fields required by the LLM client.

Provider selection

`config_from_environment()` and `create_provider()` separate configuration from construction. Supported providers are:

- `offline`: deterministic local pipeline test.
- `openai`: OpenAI Responses API.
- `gemini`: Google Gemini generateContent API.
- `anthropic`: Anthropic Messages API.
- `ollama`: local Ollama chat API.

Hosted and Ollama model names are explicit configuration values. This avoids silently selecting a model that may change availability or behavior.

Offline provider

`OfflineProvider` makes development reproducible. Grounded prompts extract a concise answer from the first approved source and append its citation. Emergency prompts route to local emergency services. Unsafe requests refuse diagnosis, treatment, and dosage advice. Missing-context prompts state that verified information is insufficient.

HTTP transport and retries

`json_request()` uses Python's standard library to send JSON. Retries are bounded and apply to connection failures and temporary HTTP statuses such as 429, 500, 502, 503, and 504. Final errors are sanitized before entering failure artifacts.

Provider adapters

Each adapter translates the same Phase 8 prompt into its provider's request shape, then calls `response_result()` to normalize text, usage, finish reason, citations, latency, and request ID.

Official API references:

- [OpenAI text generation and Responses API](#)
- [Google Gemini text generation](#)
- [Anthropic Messages API](#)
- [Ollama chat API](#)

Response safety validation

`validate_response()` rejects:

- empty outputs;
- citation labels not supplied in the prompt;
- grounded answers without a citation;
- emergency responses without emergency routing; and

- unsafe medical-advice responses without a refusal.

These checks are an important boundary, but they do not replace the dedicated Phase 11 safety guardrails or human clinical governance.

Batch generation and artifacts

`run_llm_batch()` processes all prompts independently, records successful responses, captures sanitized failures, writes JSON and CSV files, creates plots, and summarizes counts and latency.

Command-line entry point

Run the default offline workflow:

```
python 06_rag_pipeline/09_llm_client.py
```

Select a configured provider:

```
python 06_rag_pipeline/09_llm_client.py --provider openai
```

7. Notebook code sections

The notebook is an interactive explanation and validation layer:

1. Locates the project from the workspace root, project root, or 13_notebooks.
2. Imports the shared Python module dynamically.
3. Loads the environment and Phase 8 prompt bundles.
4. Displays the prompt contract without displaying credentials.
5. Generates representative grounded, emergency, and unsafe responses.
6. Executes the complete Phase 9 batch.
7. Asserts that all 12 sample prompts succeed and mode counts match expectations.
8. Displays the numbered latency and output-length plots.

The notebook deliberately uses the offline provider for executable examples. Real provider calls can cost money and may transmit data outside the hospital environment.

8. Notebook versus Python module

Topic	09_llm_client.ipynb	09_llm_client.py
Main role	Learning, inspection, demonstrations, and visual validation.	Reusable implementation and automation.
Logic source	Imports and calls the Python module.	Owns provider, validation, batching, and output logic.
Output display	Shows examples, reports, and plots inline.	Writes files and prints a concise run summary.
Execution	Run cells in order.	Run from CLI or import into Phase 10.
Provider used in validation	Forced to offline.	Uses environment or <code>--provider</code> .
Credentials	Never displayed.	Read at request time and never persisted.

The files are compatible because there is only one implementation of provider behavior: the Python module. The notebook supplies explanation around it.

9. Validation results

The included sample run uses the offline provider and validates:

- 12 input prompts;
- 12 generated responses;
- 0 failed responses;
- 9 grounded answers;
- 1 emergency response; and
- 2 unsafe-medical-advice refusals.

Grounded outputs contain only approved source citations, and no API call is made during this validation.

10. Healthcare safety and privacy

- Do not use the chatbot to diagnose disease or recommend medication dosage.
- Route possible emergencies to local emergency services immediately.
- Send only approved, necessary information to external model providers.
- Do not include protected health information unless the provider, contract, infrastructure, and workflow are authorized for it.
- Keep citations visible so patients and staff can trace answers to approved hospital material.
- Log operational metadata conservatively and redact sensitive content.
- Evaluate correctness, hallucination, bias, and safety before deployment.

Phase 9 provides a controlled model boundary. It is not clinical decision support and does not replace qualified healthcare professionals.